ЛАБОРАТОРНА РОБОТА № 2

Тема: Розгалужені та циклічні обчислювальні процеси у мові С, консольні меню та оформлення інженерної звітності Посилання на GitHub:

Мета роботи: Набути практичних навичок використання операторів розгалуження та циклів у мові С, реалізації табулювання кусочно-заданої функції, наближеного обчислення функції за допомогою нескінченного чисельного ряду та рекурентного співвідношення, а також закріпити навички роботи із системою контролю версій Git.

Етап 1. Табулювання кусочно-заданої функції

Для першого завдання відповідно до варіанта 15 було реалізовано табулювання функції: $Y(X) = 2X$, якщо $X \le 0$; $Y(X) = 1 / (X + 1)$, якщо $0 < X \le 3$; $Y(X) = \sqrt{X} - 3$, якщо $X > 3$. Програма зчитує межі відрізка Xstart, Xend та крок dX. Перед початком обчислень перевіряється, що Xstart < Xend і dX > 0. Для проходження по діапазону використовується цикл for, а вибір потрібної формули здійснюється конструкцією if - else if - else. Перед діленням перевіряється, що знаменник не дорівнює нулю, а перед добуванням кореня — що підкореневий вираз невід'ємний; в іншому разі у таблицю виводиться [Не визначено]. Область допустимих значень функції: $(-\infty; +\infty)$. Точки $X = 0$ та $X = 3$ є точками розриву першого роду: у точці $X = 0$ значення зліва дорівнює 0, а справа 1; у точці $X = 3$ значення зліва дорівнює 0,25, а справа $\sqrt{3} - 3 \approx -1{,}268$

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.F7.11.000 –Лр.2		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Чепель О.В.			Звіт з лабораторної роботи №2	Літ.	Арк. 1 / Акрушів
Перевір.		Лисенко М.С.					
Реценз.						ФІКТ, гр. КІ-26-1	
Н. Контр.							
Зав.каф.		Лобанчикова Н.М					

Лістинг програми task1.c

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    double x_start, x_end, dx, x, y;

    printf("Введіть Xstart: ");
    if (scanf("%lf", &x_start) != 1) {
    printf(" некоректна відповідь\n"); return 1;
    }
    printf("Введіть Xend: ");
    if (scanf("%lf", &x_end) != 1) {
    printf(" некоректна відповідь\n"); return 1;
    }
    printf("Введіть крок dX: ");
    if (scanf("%lf", &dx) != 1) {
    printf(" некоректна відповідь\n"); return 1;
    }

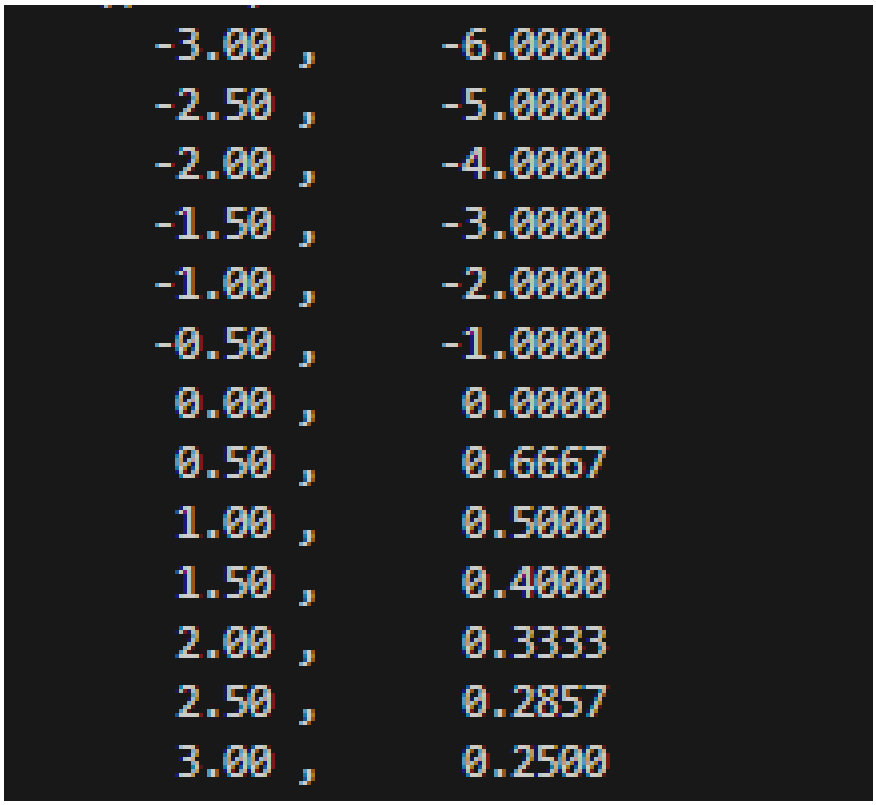
    if (x_start >= x_end) { printf("Xstart має бути більше Xend\n"); return 1; }
    if (dx <= 0)          { printf("крок має бути більше 0\n"); return 1; }


     for (x = x_start; x <= x_end; x += dx) {
        if (x <= 0) {
            y = 2 * x;
            printf("%10.2lf , %11.4lf\n", x, y);
        }
        else if (x <= 3) {
            if (x + 1 == 0) {
                printf("%10.2lf , [Не визначено]\n", x);
            }
            else {
                y = 1 / (x + 1);
                printf("%10.2lf , %11.4lf\n", x, y);
            }
        }
        else {
            if (x - 3 < 0) {
                printf("%10.2lf , [Не визначено]\n", x);
            }
            else {
                y = sqrt(x - 3);
                printf("%10.2lf , %11.4lf \n", x, y);
            }
        }
    }
    return 0;
}
```

Компіляція виконувалася командою: gcc -Wall -Wextra task1.c -o task1.exe -lm

Результати тестування:

task1.c Для перевірки правильності роботи програми було використано відрізок від -3 до 3 з кроком 0.5. Отримані значення:



При введенні від’ємного або нульового кроку програма завершує роботу з повідомленням про некоректні вхідні дані.

Етап 2. Наближене обчислення функції чисельним рядом

Для варіанта 15 використовується розклад: $\cos^2(X) = 1 - 2X^2/2! + 8X^4/4! - 32X^6/6! + \ldots$, область збіжності $-\infty < X < +\infty$. Починаючи з n = 1, поточний член ряду має вигляд $a_n = (-1)^n \cdot 2^{(2n-1)} \cdot X^{(2n)} / (2n)!$. Для рекурентного обчислення знайдено відношення двох сусідніх членів: $R(n, X) = a_n / a_{n-1} = (-1) \cdot [2^{(2n-1)} / 2^{(2n-3)}] \cdot [X^{(2n)} / X^{(2n-2)}] \cdot [(2n-2)! / (2n)!]$. Тут $2^{(2n-1)} / 2^{(2n-3)} = 4$, $X^{(2n)} / X^{(2n-2)} = X^2$, а оскільки $(2n)! = 2n \cdot (2n-1) \cdot (2n-2)!$, множник $(2n-2)!$ скорочується і залишається $1 / ((2n-1) \cdot 2n)$. Отже, $R(n, X) = -4X^2 / ((2n - 1) \cdot 2n)$, а наступний член обчислюється за формулою $a_n = a_{n-1} \cdot (-4X^2) / ((2n - 1) \cdot 2n)$. Формула діє для $n \geq 2$; перші два члени $a_0 = 1$ та $a_1 = -X^2$ задаються безпосередньо. Підсумовування виконується циклом while доти, доки $|a_n| \geq \varepsilon$. Для захисту від зациклення встановлено обмеження 100000 ітерацій. Оскільки ряд збігається при всіх X, перевіряється лише умова $\varepsilon > 0$.

Лістинг програми task2.c

```c
#include <stdio.h>
#include <math.h>

int main()
{
    double x, epsilon, sum, a, exact, pohybka;
    int n, kilkist;

    printf("введіть X: ");
    scanf("%lf", &x);
    printf("введіть точність епсілон: ");
    scanf("%lf", &epsilon);

    sum = 1;
    a = -x * x;
    n = 1;
    kilkist = 0;
    if (epsilon <= 0) {
        printf("епсілон має бути більше за 0\n");
        return 1;
    }

    while (fabs(a) >= epsilon) {
        sum = sum + a;
        n++;
        kilkist++;
        a = a * (-4 * x * x) / ((2 * n - 1) * (2 * n));
        if (kilkist > 100000) {
            printf("ряд не сбігається\n");
            break;
        }
    }

    exact = cos(x) * cos(x);
    pohybka = fabs(sum - exact);

    printf("сума ряду: %lf\n", sum);
    printf("точне значення: %lf\n", exact);
    printf("похибка: %lf\n", pohybka);
    printf("кількість ітерацій: %d\n", kilkist);

    return 0;
}
```

Компіляція виконувалася командою: gcc -Wall -Wextra task2.c -o task2.exe -lm

X	eps	значення	похибка	ітерації
1	10^{-2}	0.2919265817	$3.04 \cdot 10^{-3}$	3
1	10^{-4}	0.2919265817	$4.18 \cdot 10^{-6}$	5
1	10^{-6}	0.2919265817	$9.24 \cdot 10^{-8}$	6
3	10^{-2}	0.9800851433	$7.24 \cdot 10^{-3}$	8

Висновок

У ході лабораторної роботи було реалізовано дві програми мовою C для дослідження розгалужених і циклічних обчислювальних процесів. У першому завданні виконано табулювання кусочно-заданої функції варіанта 15 з перевіркою меж відрізка та кроку. Для вибору потрібної формули використано конструкцію if - else if - else, а перебір аргументів виконано циклом for. У другому завданні реалізовано наближене обчислення функції $\cos^2(X)$ за допомогою нескінченного степеневого ряду. Наступний член ряду обчислювався рекурентно за формулою $a_n = a_{n-1} \cdot (-4X^2) / ((2n - 1) \cdot 2n)$, що дозволило уникнути повторного обчислення степенів і факторіалів. Для завершення обчислень використовувався критерій $|a_n| < \varepsilon$ та захисний лічильник із максимальною кількістю 100000 ітерацій. Під час тестування встановлено, що зі зменшенням заданої точності ε результат стає ближчим до точного значення, однак кількість ітерацій збільшується. Також при віддаленні X від нуля ряд збігається повільніше. Файли програм і звіт підготовлено для збереження в каталозі lab02 та публікації у віддаленому репозиторії GitHub. У результаті мету лабораторної роботи досягнуто: закріплено практичні навички роботи з умовними операторами, циклами, чисельними рядами, рекурентними співвідношеннями, перевіркою вхідних даних і системою контролю версій Git.